Question 1 1 / 1 point

Which of the following operators cannot be overloaded in C++?

- ○ -
- ○ *
- ○ +
- ✓ ◉ .

Question 2 1 / 1 point

Which of the following operators can be overloaded?

- ○ .
- ○ ::
- ✓ ◉ []
- ○ ?:

Question 3 1 / 1 point

What is the correct syntax for overloading + as a member function?

- ✓ ◉ Account operator+(Account a)
- ○ Account + operator(Account a)
- ○ operator+(Account a)
- ○ operator Account+(Account a)

Question 4 0 / 1 point

When overloading an operator as a member function, the left operand must be:

- ✗ ◉ A pointer
- ➡ ○ An object of the class
- ○ An object of the class
- ○ A built-in type

Question 5 1 / 1 point

What is the purpose of the `friend` keyword in operator overloading?

- ○ To inherit operators
- ○ To make operators faster
- ✓ ◉ To allow non-member functions to access private data
- ○ To make operators faster

Question 1 1 / 1 point

What happens when you copy a unique_ptr with the keyword move()?

- ○ The pointer is duplicated
- ○ Compilation error — copy is deleted
- ✓ ◉ Ownership transfers automatically
- ○ Runtime error - not allowed

Question 2 1 / 1 point

What happens when you copy a unique_ptr **without** the keyword move()?

- ✓ (●) Compilation error — copy is not allowed
- ( ) Ownership transfers automatically
- ( ) The original is set to null pointer
- ( ) The pointer is duplicated

Question 3 1 / 1 point

What is the purpose of weak_ptr?

- ( ) To hold exclusive ownership of a resource
- ( ) To replace raw pointers in all scenarios
- ✓ (●) To observe a shared_ptr without affecting its reference count
- ( ) To automatically delete memory when out of scope

Question 4 1 / 1 point

Which function is preferred over 'new' when creating a shared_ptr?

- ✓ (●) make_shared()
- ( ) shared_ptr()
- ( ) make_unique()
- ( ) allocate_shared()

Question 5 1 / 1 point

To overload the << operator for output streams, it must be defined as:

- ✓ (●) friend function
- ( ) cousin function
- ( ) static function
- ( ) const function

Question 6 1 / 1 point

What is the return type of the prefix ++ operator overload?

- ✓ (●) T& (by reference)
- ( ) const T&
- ( ) T (by value)
- ( ) void

Question 7 1 / 1 point

What is the dummy int parameter in postfix ++ used for when a operator overloading is defined?

- ( ) It's the return value
- ( ) It represents the previous value
- ✓ (●) It distinguishes postfix from prefix overloads
- ( ) It stores the increment amount

Question 8 1 / 1 point

Which of these correctly overloads the + operator as a member function?

- ○ static MyClass operator+(MyClass a, MyClass b)
- ○ friend MyClass operator+(MyClass a)
- ✓ ◉ MyClass operator+(MyClass& b) {}
- ○ void operator+(MyClass b) {}

Question 9 1 / 1 point

How do you dynamically allocate an array of 10 ints?

- ✓ ◉ int* arr = new int[10];
- ○ int arr[] = new int(10);
- ○ int arr = new int(10);
- ○ int* arr = malloc(10);

Question 10 1 / 1 point

What is the correct way to delete a dynamically allocated array?

- ○ delete arr[];
- ✓ ◉ delete[] arr;
- ○ delete *arr;
- ○ malloc(arr);

Example Question

Q3-Assuming the definition,

class BasePlusCommissionEmployee : public CommissionEmployee
which of the following is false?

A) CommissionEmployee is the base class and BasePlusCommissionEmployee is the derived class.

B) The keyword public indicates the type of inheritance.

C) The colon (:) in the header of the class definition indicates inheritance.

D) All the public and protected members of class BasePlusCommissionEmployee are inherited as public and protected members, respectively, into class CommissionEmployee.

Example 2

Q6-Which of the following statements about inheriting base class constructors is false?

A) By default, each inherited constructor has the same access level (public, protected or private) as its corresponding base-class constructor.

B) When an object of derived class is created, first the derived class constructors are executed.

C) When a derived-class object is destroyed, the program calls that object's base destructor.

D) If the derived class does not explicitly define constructors, the compiler generates a default

Example 3

Q7-When an object of a derived class is instantiated, the _____ constructor initializes the _____ members.

A) Base class, derived class

B) Derived class, base class

C) Derived class, public

D) Base class, base class

Question 1 0 / 1 point

What is the result of the code:

C++

```cpp
#include <iostream>
#include <string>
using namespace std;
class A
{
    int a, b;
    float d;
   public:
    void change(int i){
        a = i;
    }
    void value_of_a(){
        cout<<a;
    }
};

class B: private A
{

};

int main(int argc, char const *argv[])
{
    B b;
    cout<<sizeof(B);
    return 0;
}
```

- ○ 8
- ○ Runtime Error
- ⇨ ○ 12
- ✖ ◉ Compilation Error

Question 2 1 / 1 point

Which specifier makes all the data members and functions of base class inaccessible by the derived class?

- ○ both public and protected
- ○ protected
- ✓ ◉ private
- ○ public

Question 3 0 / 1 point

If a class is derived privately from a base class then

- ○ no derivation of the class gives an error
- ○ no members of the base class is inherited
- ➡ ○ all the members are inherited by the class but are hidden and cannot be accessible
- ✗ ◉ all members are accessible by the derived class

Question 4 1 / 1 point

What is the correct inheritance syntax?

- ○ class B inherits A {}
- ○ class B extends A {}
- ✓ ◉ class B : public A {}
- ○ class B -> A {}

Question 5 1 / 1 point

What does protected inheritance do to inherited data members and functions?

- ○ No change
- ○ Makes everything public
- ○ Makes public → private
- ✓ ◉ Makes public → protected

Question 6

1 / 1 point

What is the output of the code block?

JavaScript

```
#include <iostream>
#include <string>
using namespace std;
class A
{
    float d;
   public:
    A(){
        cout<<"Constructor of class A\n";
    }
};

class B: public A
{
    int a = 15;
    public:
    B(){
        cout<<"Constructor of class B\n";
    }
};

int main(int argc, char const *argv[])
{
    B b;
    return 0;
}
```

- ○ `Constructor of class A`
- ○ `Constructor of class B`
- ○ `Constructor of class B`
  `Constructor of class A`
- ✓ ◉ `Constructor of class A`
  `Constructor of class B`

Question 7

1 / 1 point

Which is TRUE about private inheritance?

- ○ Protected becomes public
- ✓ ◉ Public members become private
- ○ No inheritance occurs
- ○ Public members stay public

Question 8

0 / 1 point

What is the order of execution?

C++

```cpp
class A {
public:
    A() { cout << "A "; }
    ~A() { cout << "~A "; }
};

class B : public A {
public:
    B() { cout << "B "; }
    ~B() { cout << "~B "; }
};

int main() {
    B obj;
}
```

- ○ B A ~B ~A
- ➡ ○ A B ~B ~A
- ○ B A ~A ~B
- ✗ ◉ A B ~A ~B

Question 9

0 / 1 point

What is the result of the code?

```cpp
class A {
public:
    void show() { cout << "A"; }
};

class B : public A {};
class C : public A {};

class D : public B, public C {};

int main() {
    D obj;
    obj.show();
}
```

- ○ Undefined
- ○ Prints A
- ✖ ◉ Runtime error
- ➡ ○ Compilation error

Question 10

1 / 1 point

What happens if a derived class redefines a function that is already defined in its base case in C++?

- ○ Both are available
- ○ Compilation error
- ✔ ◉ function in the derived class overrides the function in base class
- ○ function in the base class overrides the function in derived class

Question 1 (10 points)

Write a C++ program to find the minimum value in a 3 × 4 two-dimensional array using recursion.

Requirements:

1. Define an abstract base class called ArrayProcessor that contains:
    - A pure virtual function: virtual int findMin;
2. Create a derived class called MinFinder that:
    - Inherits base class ArrayProcessor
    - Implements the findMin function using recursion to traverse the array and determine the minimum value
3. The recursive function should:
    - visit each element of the array exactly once
4. In the main() function:
    - Declare and initialise a 3x4 with sample values
    - array<array<int, 4>, 3> sample_array = {{

      {5, 2, 9, 1},

      {8, 7, 3, 4},

      {6, 0, 11, -2}

      }};
    - Create an object of the derived class MinFinder
    - Use a base class pointer ArrayProcessor*, to call the findMin function
    - Display the minimum value using cout.

Question 2 (1 point)

Rewrite the below code using inheritance, polymorphism and smart ptr.

C++

```cpp
#include <iostream>
using namespace std;

class Rose {
    public:
        Rose(string name): name{name} {}
        void draw() {
            cout << "Drawing a rose" << endl;
        }
    private:
        string name;
};

class Tulip {
    public:
        Tulip(string name): name{name} {}
        void draw() {
            cout << "Drawing a tulip" << endl;
        }
    private:
        string name;
};

class SunFlower {
    public:
        SunFlower(string name): name{name} {}
        void draw() {
            cout << "Drawing a sunflower" << endl;
        }
    private:
        string name;
};

enum FlowerType {ROSE, TULIP, SUNFLOWER};

struct Flower {
    FlowerType type;
    shared_ptr<void> flower;
};

int main() {
    vector<Flower> flowerObjects;
    int user_choice{0};

    while(true) {
        cout << "Enter 1 for Rose, 2 for Tulip, 3 for Sunflower" << endl;
        cin >> user_choice;

        if (user_choice != 1 && user_choice != 2 && user_choice != 3) {
            break;
        }

        switch(user_choice) {
            case 1:
                flowerObjects.push_back({ROSE, make_shared<Rose>("this is rose")});
                break;
            case 2:
                flowerObjects.push_back({TULIP, make_shared<Tulip>("this is tulip")})
                break;
            case 3:
                flowerObjects.push_back({SUNFLOWER, make_shared<SunFlower>("this is s
                break;
            default:
                break;
        }
    }

    for(auto flower : flowerObjects) {
        switch(flower.type) {
            case ROSE:
                static_cast<Rose*>(flower.flower.get())->draw();
                break;
            case TULIP:
                static_cast<Tulip*>(flower.flower.get())->draw();
                break;
            case SUNFLOWER:
                static_cast<SunFlower*>(flower.flower.get())->draw();
                break;
            default:
                break;
        }
    }

    return 0;
}
```

Question 2 (1 point)

Rewrite the below code using inheritance, polymorphism and smart ptr.

C++

```
 <iostream>
mespace std;

se {
ic:
Rose(string name): name{name} {}
void draw() {
    cout << "Drawing a rose" << endl;
}
ate:
string name;


lip {
ic:
Tulip(string name): name{name} {}
void draw() {
    cout << "Drawing a tulip" << endl;
}
ate:
string name;


nFlower {
ic:
SunFlower(string name): name{name} {}
void draw() {
    cout << "Drawing a sunflower" << endl;
}
ate:
string name;


werType {ROSE, TULIP, SUNFLOWER};

lower {
erType type;
ed_ptr<void> flower;


() {
or<Flower> flowerObjects;
user_choice{0};

e(true) {
cout << "Enter 1 for Rose, 2 for Tulip, 3 for Sunflower" << endl;
cin >> user_choice;

if (user_choice != 1 && user_choice != 2 && user_choice != 3) {
    break;
}

switch(user_choice) {
    case 1:
        flowerObjects.push_back({ROSE, make_shared<Rose>("this is rose")});
        break;
    case 2:
        flowerObjects.push_back({TULIP, make_shared<Tulip>("this is tulip")});
        break;
    case 3:
        flowerObjects.push_back({SUNFLOWER, make_shared<SunFlower>("this is sunflower")});
        break;
    default:
        break;
}


auto flower : flowerObjects) {
switch(flower.type) {
    case ROSE:
        static_cast<Rose*>(flower.flower.get())->draw();
        break;
    case TULIP:
        static_cast<Tulip*>(flower.flower.get())->draw();
        break;
    case SUNFLOWER:
        static_cast<SunFlower*>(flower.flower.get())->draw();
        break;
    default:
        break;
}


rn 0;
```

Question 1 (1 point)

You are working on a project in a team, and while reviewing the code of your colleague, you saw the keyword **virtual ~className() = default;**

Is this keyword important, and what happens when it is missing?

Question 2 (1 point)

When working with inheritance, does it make sense to define a destructor?. Where should this be defined?

Question 3 (1 point)

Convert this code to an abstract class.

```cpp
class Employee {
public:
    double earnings() const {
        return 0.0;
    }
};
```

Question 4 (1 point)

Identify and class the bugs, rewrite the solution for the affected parts only.

C++

```cpp
#include <iostream>
#include <memory>
using namespace std;

class FirstClass {
public:
    void func1() {
        cout << "FirstClass func1\n";
    }

    void func2() {
        cout << "FirstClass func2\n";
    }
};

class SecondClass : public FirstClass {
public:
    void func1() override {
        cout << "SecondClass func1\n";
    }

    void func2() const {
        cout << "SecondClass func2\n";
    }
};

int main() {
    unique_ptr<FirstClass> p1 = make_unique<SecondClass>();

    FirstClass obj = SecondClass();

    p1->func1();

    return 0;
}
```

Question 5 (1 point)

If a derived class does not implement all of the pure virtual methods of its parent, then it becomes a type of its parent class?

- ◯ True
- ◯ False

Question 6 (1 point)

A recruiter asked you to differentiate between an " is-a" and "has-a" relationship in C++. How do you discuss this, and when do you use either?

Question 7 (1 point)

Does inheritance always improve code reuse, or can it sometimes make systems harder to maintain?

Question 8 (1 point)

An AI generated the code below, and while reviewing, do you think the approach makes sense? If you were to redo this, what would you change? and why?

```cpp
#include <iostream>
#include <string>
using namespace std;

class Shape {
public:
    double width;
    double height;
};

class Rectangle {
public:
    Rectangle(double w, double h) {
        shape.width = w;
        shape.height = h;
    }

    double area() {
        return shape.width * shape.height;
    }

private:
    Shape shape;
};
```

Question 9 (1 point)

Polymorphism is implemented via:

- ◯ Non-virtual functions.
- ◯ Member functions.
- ◯ virtual functions and dynamic binding.
- ◯ inline functions.

Question 10 (1 point)

Complete the following code as stated by the command.

```cpp
#include <iostream>
class Shape {
public:
    virtual void draw() { std::cout << "Drawing a Shape." << std::endl; } };

class Square : public Shape {
public:
    void draw() override {  std::cout << "Drawing a Square." << std::endl;  } };

int main() {
    complete this command // Pointer (squarePtr) to the base class
    Shape shapeObject;   // Object of type Shape
    Square squareObject; // Object of type Square

    complete this command
    squarePtr->draw(); // Calls Shape's draw() at runtime using polymorphism

    complete this command
    squarePtr->draw(); // Calls Square's draw() at runtime using polymorphism
}
```

Question 11 (1 point)

The line: virtual double earnings() const = 0; appears in a class definition. You **cannot** deduce that:

- ◯ This class is an abstract class.
- ◯ All classes that directly inherit from this class will override this method.
- ◯ Any concrete class derived from this class will have an earnings function.
- ◯ This class will probably be used as a base class for other classes.

Question 12 (1 point)

What can be concluded about class shape?

```cpp
class Shape {
    public:
        virtual double area() const = 0;
        virtual void draw() const = 0;
        virtual ~Shape() {}
};
```

- ◯ The destructor of the derived class is pure virtual.
- ◯ All derived classes must override the area and draw functions.
- ◯ Objects of the Shape class cannot be instantiated.
- ◯ The destructor of the Shape class is pure virtual.